Universidade do Minho

Unidade Curricular de Sistemas Operativos 2025/2026

Relatório de Projeto

Bruno Freitas - A110016 , **João Ribeiro - A111041** , **Tiago Santos - A110313**

Data de Entrega: 10 de Maio de 2026

1 Introdução

No contexto da unidade curricular de Sistemas Operativos, foi proposto o desenvolvimento de um ambiente de orquestração de execução de comandos, com o objetivo de aplicar conhecimentos sobre processos, comunicação entre processos e manipulação de ficheiros no sistema operativo Linux.

O sistema é composto por dois componentes principais: um controlador (controller), responsável por gerir e escalonar os pedidos de execução recebidos, e um executor (runner), que permite ao utilizador submeter comandos e consultar o estado do sistema através da linha de comandos. A comunicação entre ambos é realizada exclusivamente através de pipes com nome (named pipes / FIFOs), recorrendo a chamadas ao sistema fornecidas pelo Linux.

Entre as funcionalidades implementadas, destacam-se a submissão e escalonamento de comandos, a consulta de comandos em execução e em espera, e o registo persistente da atividade do sistema. Para além destas funcionalidades base, o projeto inclui também suporte a operadores da linha de comandos, nomeadamente redirecionamento e pipes entre processos, execução paralela de múltiplos comandos com um grau de paralelismo configurável, e avaliação experimental de diferentes políticas de escalonamento.

Este relatório descreve a arquitetura do sistema, as decisões de implementação adotadas, os mecanismos de comunicação utilizados e os testes realizados, incluindo uma análise comparativa do impacto das políticas de escalonamento e das configurações de paralelismo na experiência dos utilizadores.

2 Esquema Ilustrativo

3 Protocolo de Comunicação

O sistema segue um modelo *cliente-servidor* baseado em *pipes* com nome (FIFOs). O `controller` é o servidor — único e persistente — e cada `runner` é um cliente efémero que envia um pedido, aguarda a resposta e termina. A escolha de FIFOs justifica-se pela simplicidade, pela persistência no sistema de ficheiros e por terem sido o mecanismo abordado em aula.

3.1 Topologia das FIFOs

O protocolo usa duas categorias de FIFO:

- **FIFO público** (`/tmp/controller_fifo`): criado pelo `controller` no arranque, é o ponto de entrada único do sistema. Todos os runners escrevem aqui, e o servidor lê continuamente. É aberto em `O_RDWR` para evitar que o descritor feche quando não há nenhum runner ligado.
- **FIFO privado por runner** (`/tmp/runner_<pid>_fifo`): criado pelo próprio runner, com base no seu PID. Serve para o servidor responder àquele runner em particular, evitando que respostas se cruzem entre clientes.

3.2 Estrutura `Message`

Toda a comunicação assenta numa `struct` de tamanho fixo definida em `protocol.h`, com os campos: `runner_pid`, `user_id`, `command_id`, `command[256]`, `msg_type` e `start_time`. O tamanho fixo permite que cada `read()` devolva exactamente uma mensagem, sem delimitadores nem parsing. Como `sizeof(Message)` é muito inferior a `PIPE_BUF` (4096 bytes em Linux), o POSIX garante que cada `write()` é atómico — mensagens de runners diferentes nunca se entrelaçam no FIFO público.

3.3 Tipos de mensagem

O campo `msg_type` distingue cinco operações:

- `MSG_EXECUTE` — submissão de um comando (`-e`).
- `MSG_DONE` — runner notifica o fim da execução.
- `MSG_STATUS` — pedido do estado das filas (`-c`).
- `MSG_SHUTDOWN` — pedido de encerramento (`-s`).
- `MSG_STATUS_DONE` — mensagem interna do servidor para recolha de zombies (ver 3.5).

3.4 Fluxos de comunicação

Submissão (`-e`). O runner cria o seu FIFO privado e abre-o em `O_RDWR` *antes* de contactar o servidor. Esta abertura prévia elimina uma *race condition*: quando o controller mais tarde chamar `open(O_WRONLY)` para responder, já há um leitor registado. O runner escreve então a `Message` no FIFO público; o controller autoriza imediatamente (`"OK"` no FIFO privado) ou enfileira o pedido. Após executar o comando — incluindo *pipelines* e redirecionamentos — o runner envia `MSG_DONE`, e o controller regista a duração no `controller.log` e promove o próximo da fila.

Consulta (`-c`). O controller faz `fork()` para o filho compor a resposta sem bloquear o loop principal. O filho escreve as listas `--Executing` e `--Scheduled` no FIFO privado do runner e termina (ver 3.5).

Encerramento (`-s`). O controller marca `shutdown_requested` e deixa de aceitar novos `MSG_EXECUTE`, mas continua a tratar os jobs em curso. Quando a fila de execução fica vazia, faz `unlink()` ao FIFO público e confirma ao runner que pediu o shutdown.

3.5 Recolha dos filhos do `-c`

Cada `fork()` para tratar um `-c` cria um processo que, ao chamar `exit()`, fica em estado *zombie* até ser recolhido. Em vez de `signal(SIGCHLD, SIG_IGN)` — fora do âmbito da disciplina — desenhámos um protocolo interno: antes de `exit()`, o filho envia `MSG_STATUS_DONE` ao pai pelo FIFO público com o seu PID. O loop principal, ao receber a mensagem, chama `waitpid()` com esse PID. Como o filho já fez `exit` antes da mensagem ser lida, o zombie já existe e o `waitpid` recolhe-o sem bloquear.

3.6 Vantagens da abordagem

- **Concorrência controlada** pelo limite `parallel-tasks`, sem deixar de aceitar novos pedidos.
- **Isolamento entre clientes** via FIFO privado nomeado pelo PID.
- **Mensagens atómicas** graças ao tamanho fixo abaixo de `PIPE_BUF`.
- **Recolha de zombies sem** `signal`, usando apenas FIFOs e `waitpid`.
- **Robustez a *race conditions*** pela abertura prévia em `O_RDWR` dos FIFOs.

4 Decisões de Implementação

Durante o desenvolvimento do projeto, foram tomadas várias decisões arquiteturais para garantir a simplicidade, robustez e eficiência do sistema:

- **Alocação estática das filas de espera e execução:** Optou-se por utilizar *arrays* estáticos (`executing[MAX_JOBS]` e `scheduled[MAX_JOBS]`) com um limite máximo generoso (1024), em detrimento de estruturas de dados dinâmicas, como listas ligadas. Esta abordagem simplifica a gestão de memória (eliminando chamadas a `malloc` e `free`) e anula a probabilidade de *memory leaks* e falhas de segmentação na gestão de múltiplos pedidos simultâneos.
- **Identificador único do comando (`command_id`):** O identificador único de comando, exigido pelo enunciado, é gerado pelo `runner` com base no seu próprio PID (`getpid()`). Como o sistema operativo garante a unicidade do PID de processos em execução, evitou-se a necessidade de sincronização complexa entre múltiplos clientes para atribuição de IDs.
- **Prevenção de *busy waiting* no servidor:** O `controller` abre o FIFO público com a flag `O_RDWR` (leitura e escrita), em vez de apenas `O_RDONLY`. Caso fosse aberto apenas para leitura, o `read()` retornaria EOF (0) sempre que o último `runner` terminasse, resultando num ciclo infinito de consumo de CPU. Ao manter uma extremidade de escrita aberta, a chamada bloqueia eficientemente até à chegada de um novo cliente.
- **Política Round-Robin baseada no Utilizador:** A política alternativa de escalonamento foi desenhada para garantir a justiça no *throughput*. Ao selecionar o próximo processo da fila, o escalonador procura iterativamente o primeiro comando pertencente a um utilizador diferente daquele que foi escalonado na iteração anterior. Se a fila contiver apenas comandos de um único utilizador, a política degenera suavemente para FCFS, garantindo a ausência de inanição (*starvation*).

5 Implementações e Funcionalidades

5.1 Operações do Cliente (`runner`)

O programa `runner` é o ponto de interação com o utilizador e suporta três modos de operação estipulados: execução (`-e`), consulta (`-c`) e encerramento (`-s`).

No modo de execução, o principal desafio tecnológico reside no processamento de comandos complexos. A função `parse_command()` divide a *string* de entrada (fornecida pelo utilizador) em *tokens*, agrupa esses *tokens* em segmentos separados pelo operador de *pipe* (`|`), e deteta operadores de redirecionamento (`>`, `2>`, `<`), preparando matrizes de estruturas `Segment`.

Para a execução (`exec_pipeline()`), se o comando possuir N segmentos, o cliente cria $N - 1$ *pipes* anónimos. Em seguida, invoca `fork()` N vezes. Cada processo filho utiliza `dup2()` para:

- Ligar o `STDOUT` ao extremo de escrita do seu *pipe* e o `STDIN` ao extremo de leitura do *pipe* anterior.
- Sobrepor os descritores adequados caso o utilizador tenha especificado ficheiros de redirecionamento, os quais têm precedência sobre as ligações dos *pipes*.

Por fim, cada filho executa `execvp()`. O processo pai aguarda (`wait()`) que todos os componentes da *pipeline* terminem antes de notificar o servidor do sucesso da operação com a mensagem `MSG_DONE`.

5.2 Gestão do Servidor (`controller`)

O `controller` centraliza o orquestramento, garantindo que o número de processos a executar em simultâneo nunca ultrapassa o limite `parallel-tasks`.

O estado é mantido em duas filas: execução (`executing`) e espera (`scheduled`). O fluxo de gestão funciona por eventos:

- **Submissão:** Num pedido `MSG_EXECUTE`, se o limite de concorrência não tiver sido atingido, o servidor autoriza a execução via FIFO privado e insere o comando na fila `executing`. Se o sistema estiver lotado, insere o comando na fila `scheduled`.
- **Término e Registo:** Ao receber um `MSG_DONE`, o servidor calcula o tempo total de resposta (usando `gettimeofday()`) e regista-o persistentemente no ficheiro `controller.log`, juntamente com o `user_id` e o `command_id`. Depois de limpar a vaga na fila de execução, aplica a política de escalonamento escolhida (FCFS ou RR) para promover o próximo comando da fila `scheduled`, reativando o ciclo.

6 Testes e Resultados

6.1 Análise das Políticas de Escalonamento (FCFS vs RR)

6.2 Impacto do Paralelismo

7 Conclusão